# Verified Personnel Facial Classification

Samuel Ferreira, Chase Michael, Michael Kingsley, Ashton Dong

# Objective

**Core Goal:**

Using machine learning we want to create a facial classification model that can identify a known set of people.

**Potential Expansion:**

This facial recognition classification could be expanded to many use cases that require validation for entry such as unlocking the door to a house when a resident approaches.

# Pipeline Flow

### 1. Data Intake

Download LFW Faces dataset as the raw input for the pipeline.

### 2. Preparation

Split LFW & Known Faces into Train/Validation sets.

*Training: 70% | Valid: 30%*

### 3. Embedding

Extract 128-D Vectors from faces and store as .pkl files.

### 4. Training

Train classifier and store trained model (classifier.pkl).

### Live Test

Runtime Recognition

# Datasets

## Labeled faces in the wild (LFW):

- 13,233 images of 5,749 people
- Labeled directory of each person with several images

Jennifer Aniston from LFW Dataset

## Pictures of us:

In order to train the classification to identify specific individuals we needed to train it on self-collected data

# Training-Validation Split

**Training Set: ~70% of images**

- Used to create face embeddings gallery
- Used to train classification models

**Validation Set: ~30% of images**

- Used to evaluate model performance
- Ensures unbiased accuracy measure

# Face Detection Models

## HOG (Histogram of Oriented Gradients)

**Pros:**

- Very fast on CPU
- Low computational overhead
- Good accuracy on frontal faces

**Cons:**

- Less robust to extreme angles
- Requires good lighting

## CNN (Convolutional Neural Network)

**Pros:**

- More robust to varied angles
- Better with difficult lighting
- Higher accuracy overall

**Cons:**

- Computationally expensive
- Requires GPU for reasonable speed
- Not suitable for low-end hardware

# Facial Embedding

**Each face has a single 128-dimension vector which describes facial features**

**How Embeddings are Generated:**

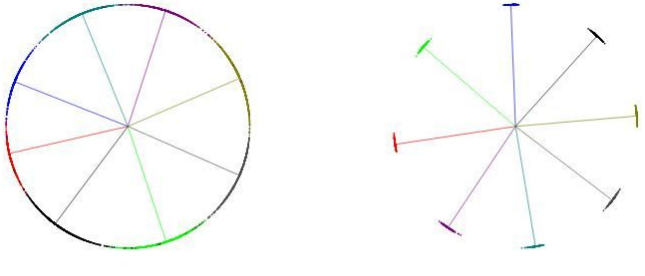
1. Load image from disk
2. Detect face location using selected backend (HOG or CNN)
3. Extract face region
4. Run through face_recognition embedding model
5. Output: 128-dimensional vector

*Distance between these vectors represents the similarity between faces*

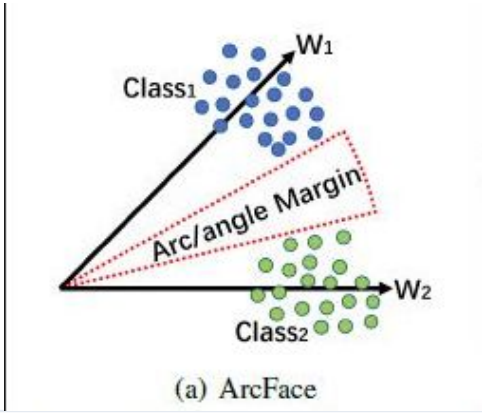
# What is ArcFace?

ArcFace is a way of calculating loss such that results for similar things are pushed closer together.

ArcFace uses a cosine similarity with a hypersphere with x dimensions in the sphere, where normalized vectors will sit on. It has two hyperparameters (s - scale, m – margin), which determine the final result.



(a) Softmax (b) ArcFace




(a) ArcFace

# Our CNN's Pipeline Flow

### 1. Download Training Data

Uses a subset of VGGFaces2 as the training dataset

VGGFaces2 has a total of 3.3M faces/images across about ~9,000 identities

### 2. Preparation

Split training data into training, validation, and testing

*Training: 80% | Valid: 10% | Testing: 10%*

### 3. Model Training

Using the ArcFace loss function to update weights in our CNN.

This represents how the faces will be encoded into embeddings.

### 4. Encoding

Faces are converted into an 512-d embedding using the model trained from the previous step.

Known faces are saved into a db file.

### 5. Testing

Model is tested against the training dataset, results for accuracy are output.

Manual images are also be fed into the model to see the output.

### Live Test

Runtime Recognition

# Facial Classification

## 1. KNN (k=3)

Instance-based learning classifies by voting among 3 nearest neighbors.

- **Strengths:** Simple, interpretable, baseline model
- **Weaknesses:** Can be slow with large datasets

## 2. Logistic Regression

Linear multiclass classifier using probabilistic decision boundaries.

- **Strengths:** Fast, stable, interpretable
- **Weaknesses:** Limited to linear boundaries

## 3. LinearSVC

Finds optimal linear separation in the 128D embedding space.

- **Strengths:** Strong for high-dimensional data
- **Weaknesses:** Requires careful regularization

## 4. Random Forest

Ensemble of decision trees that captures non-linear patterns.

- **Strengths:** Handles complexity well
- **Weaknesses:** Prone to overfitting on small datasets

# Classification Model Comparison

| Model | Train Accuracy | Test Accuracy | Train Time (s) |
|---|---|---|---|
| KNN(k=3) | 1.0000 | 0.9914 | 0.00 |
| LogisticRegression | 0.9730 | 0.9483 | 1.81 |
| **LinearSVC** | **1.0000** | **1.0000** | **0.17** |
| RandomForest | 1.0000 | 0.9828 | 0.56 |

# Tolerance Hyperparameter

## Definition & Implementation

Running face recognition on an image requires a precise tolerance value to balance sensitivity and accuracy.

**Concept:** Tolerance is the maximal acceptable difference between an image's live encoding and a trained encoding.

**Range:** Values range from 0.0 (strict) to 1.0 (loose).

**Optimization:** Our script tests validation across multiple values to determine the optimal threshold.

# Model Statistics

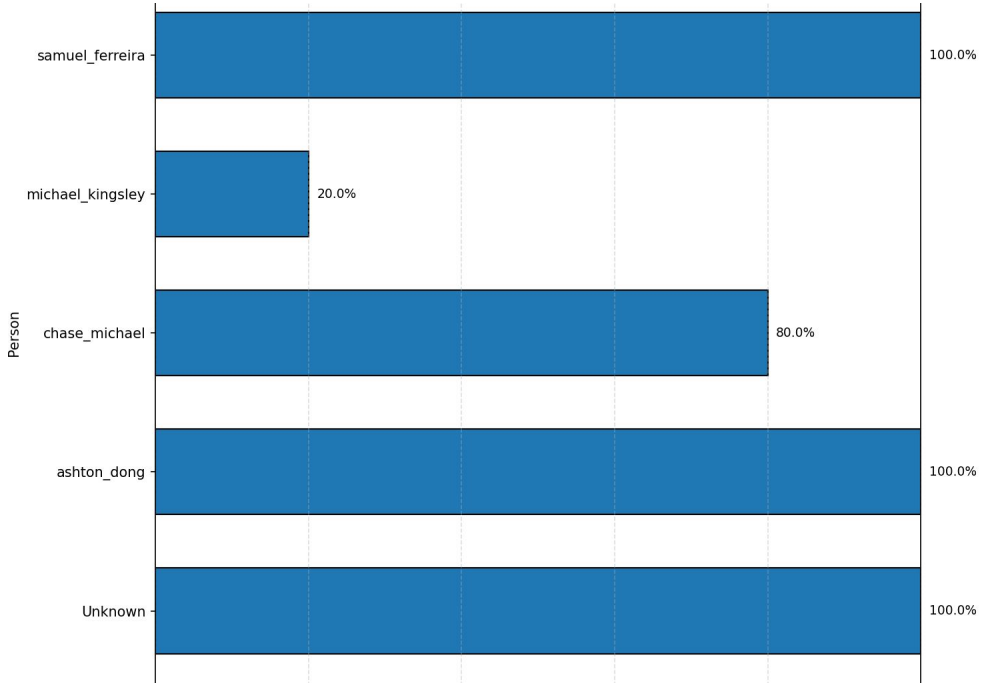
## Performance Overview

The statistics of our models performance were created using a tolerance of 0.45 and linear SVC classifier, using a random assortment of photos we provided for training.

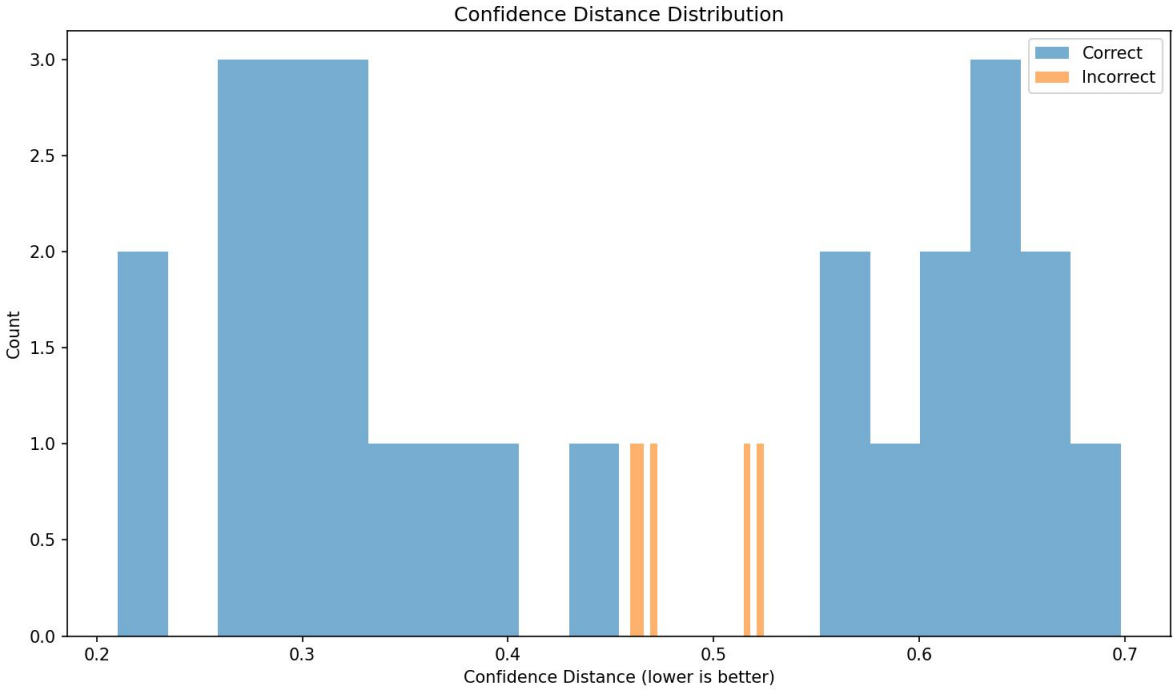
### Overall Evaluation

The model performed well overall. Michael was an obvious outlier due to extreme angles and poor lighting in the randomly selected validation pictures.

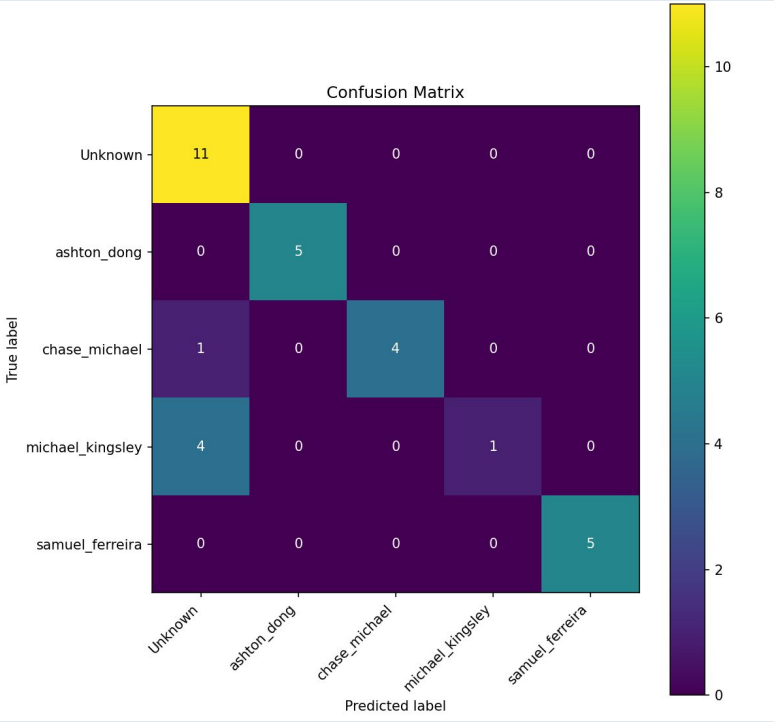
# Per-person recognition accuracy



# Confidence Histogram



# Confusion Matrix



# Acknowledgements


### Labelled Faces in the Wild Dataset

https://www.kaggle.com/datasets/jessicali9530/lfw-dataset

### Face_recognition Library

https://github.com/ageitgey/face_recognition

### ArcFace

https://arxiv.org/pdf/1801.07698.pdf

### VggFaces2

https://www.robots.ox.ac.uk/~vgg/data/vgg_face2/